

CENTRO UNIVERSITÁRIO AFYA ITAPERUNA

PROJETO DE BANCO DE DADOS RELACIONAL - ETAPA 1

EDUARDA G. MONTEIRO

ITAPERUNA

2026

SUMÁRIO

DESCRIÇÃO DO CENÁRIO	2
ESTRUTURA DO BANCO DE DADOS	4
Tabela usuario	4
Tabela artista	4
Tabela genero	4
Tabela album	5
Tabela musica	5
Tabela playlist	5
Tabela playlist_musica	5
Tabela historico_reproducao	5
RELACIONAMENTOS	6
Relacionamento entre Usuário e Assinatura (1:1)	7
Relacionamento entre Artista e Álbum (1:N)	7
Relacionamento entre Álbum e Música (1:N)	7
Relacionamento entre Gênero e Música (1:N)	7
Relacionamento entre Usuário e Playlist (1:N)	7
Relacionamento entre Playlist e Música (N:N)	8
Relacionamento entre Usuário, Música e Histórico de Reprodução	8
DECISÕES TÉCNICAS	8
Chaves Primárias	9
Chaves Estrangeiras	9
Tipos de Dados	9
Utilização de Campos NOT NULL	9
Restrições UNIQUE	10
Valores Padrão (DEFAULT)	10
Restrições CHECK	10
Chave Primária Composta	11
ESTRATÉGIA DE POPULAÇÃO	11
CONSULTAS DESENVOLVIDAS	13
Consulta 1: Músicas mais reproduzidas	14
Consulta 2: Artistas com mais músicas cadastradas	14

Consulta 3: Quantidade de músicas por gênero musical _____	14
Consulta 4: Playlists com maior quantidade de músicas _____	14
Consulta 5: Usuários sem assinatura cadastrada _____	15
Consulta 6: Músicas nunca reproduzidas _____	15
Consulta 7: Artistas com músicas reproduzidas _____	15
Consulta 8: Usuários mais ativos _____	15
Consulta 9: Músicas com reproduções acima da média _____	15
Consulta 10: Usuários com playlists contendo músicas do gênero Pop _____	16
Consulta 11: Artistas presentes em playlists _____	16
Consulta 12: Ranking dos artistas mais reproduzidos _____	16
Consulta 13: Evolução das reproduções por usuário _____	16
Consulta BÔNUS _____	17
EXPLICAÇÃO DETALHADA DAS CONSULTAS MAIS COMPLEXAS _____	17
Consulta 9: Músicas com reproduções acima da média geral _____	18
Consulta 10: Usuários que criaram playlists com músicas do gênero Pop _____	18
Consulta 12: Ranking dos artistas mais reproduzidos _____	19
Consulta 13: Evolução das reproduções por usuário _____	20
Consulta BÔNUS _____	21
CONCLUSÃO _____	22

DESCRIÇÃO DO CENÁRIO

LINK GIT: <https://github.com/eduardaguimaraess/soundwave-banco-de-dados>

A SoundWave é uma plataforma digital de streaming musical desenvolvida para permitir o gerenciamento e a distribuição de conteúdo musical de forma organizada e eficiente. Inspirada em serviços amplamente utilizados no mercado, a plataforma foi projetada para armazenar informações sobre usuários, artistas, álbuns, músicas, gêneros musicais, playlists, assinaturas e histórico de reproduções, possibilitando tanto a reprodução de músicas quanto a análise do comportamento dos usuários. O sistema foi modelado utilizando um banco de dados relacional implementado no MySQL 8.0.

O principal problema resolvido pela plataforma é o controle centralizado do catálogo musical e das interações realizadas pelos usuários. Sem um banco de dados estruturado, seria difícil gerenciar informações sobre músicas, artistas e playlists, além de acompanhar métricas importantes como quantidade de reproduções, popularidade dos artistas e nível de atividade dos usuários. O banco de dados permite armazenar essas informações de forma consistente e organizada, garantindo integridade e facilidade de acesso aos dados.

Os principais usuários do sistema são:

- **Usuários da plataforma**, responsáveis por ouvir músicas, criar playlists e consumir conteúdo musical.
- **Artistas**, que possuem álbuns e músicas cadastradas no catálogo da plataforma.
- **Administradores**, encarregados de gerenciar as informações do sistema e analisar dados para apoiar decisões relacionadas ao serviço.

Entre as operações mais relevantes realizadas pelo sistema estão o **cadastro de usuários**, o **gerenciamento de assinaturas**, o **cadastro de artistas**, **álbuns e músicas**, a **criação de playlists**, a **associação de músicas às playlists** e o **registro do histórico de reproduções**. Essas operações permitem não apenas o funcionamento básico da plataforma, mas também a geração de relatórios e análises sobre o consumo musical dos usuários.

Além das funcionalidades operacionais, a plataforma possibilita a obtenção de **informações estratégicas por meio de consultas SQL**, permitindo identificar **músicas mais reproduzidas, artistas mais populares, gêneros musicais mais consumidos, usuários mais ativos e playlists com maior quantidade de músicas**. Dessa forma, os dados armazenados podem ser utilizados para apoiar a tomada de decisões e melhorar a experiência dos usuários dentro da plataforma.

ESTRUTURA DO BANCO DE DADOS

O banco de dados da plataforma SoundWave foi desenvolvido utilizando o **SGBD MySQL 8.0** e é composto por nove tabelas principais, responsáveis por armazenar e organizar as informações necessárias para o funcionamento de um serviço de streaming musical. Cada tabela representa uma entidade do sistema e possui relacionamentos que garantem a integridade e a consistência dos dados.

Tabela usuario

Armazena os dados dos usuários cadastrados na plataforma, incluindo nome, e-mail, data de cadastro e situação da conta. Essa tabela é fundamental para identificar os usuários que utilizam o sistema e se relaciona com playlists, assinaturas e histórico de reproduções.

Tabela assinatura

Responsável pelo controle dos planos contratados pelos usuários. Contém informações como tipo de plano, valor mensal, data de início e status da assinatura. Possui relacionamento do tipo 1:1 com a tabela usuario, garantindo que cada usuário tenha no máximo uma assinatura cadastrada.

Tabela artista

Armazena os artistas presentes no catálogo da plataforma, registrando informações como nome artístico, país de origem e data de cadastro. Cada artista pode possuir um ou mais álbuns vinculados ao seu cadastro.

Tabela genero

Contém os gêneros musicais utilizados para classificar as músicas da plataforma, como Pop, Rock, Sertanejo, MPB e outros estilos musicais. Essa classificação facilita consultas e análises relacionadas às preferências dos usuários.

Tabela album

Armazena os álbuns musicais cadastrados no sistema. Cada álbum pertence a um único artista e possui informações como título e data de lançamento. Essa tabela estabelece o relacionamento entre artistas e músicas.

Tabela musica

Responsável pelo armazenamento das músicas disponíveis na plataforma. Cada música está associada a um álbum e a um gênero musical, além de possuir informações como título e duração em segundos.

Tabela playlist

Armazena as playlists criadas pelos usuários. Cada playlist pertence a um único usuário e registra informações como nome e data de criação. Um usuário pode criar várias playlists ao longo do tempo.

Tabela playlist_musica

Tabela associativa criada para representar o relacionamento muitos-para-muitos (N:N) entre playlists e músicas. Sua função é registrar quais músicas pertencem a cada playlist e a data em que foram adicionadas.

Tabela historico_reproducao

Armazena todas as reproduções realizadas pelos usuários da plataforma. Cada registro contém o usuário que reproduziu a música, a música reproduzida e a data e hora da reprodução. Essa tabela é essencial para análises estatísticas e consultas relacionadas ao comportamento dos usuários e à popularidade do catálogo musical.

De forma geral, a estrutura do banco foi projetada para representar os principais processos de uma plataforma de streaming musical, permitindo o gerenciamento eficiente de usuários, conteúdo musical e informações geradas pelas interações realizadas dentro do sistema. Os relacionamentos entre as tabelas garantem a integridade dos dados e possibilitam a execução de consultas analíticas e operacionais de diferentes níveis de complexidade.

RELACIONAMENTOS

A modelagem do banco de dados da plataforma SoundWave foi desenvolvida com base nos principais relacionamentos existentes em um serviço de streaming musical. Foram utilizados relacionamentos dos tipos 1:1 (um para um), 1:N (um para muitos) e N:N (muitos para muitos), garantindo uma representação adequada das regras de negócio e evitando redundância de informações.

Relacionamento entre Usuário e Assinatura (1:1)

O relacionamento entre as tabelas usuario e assinatura é do tipo um para um (1:1). Cada usuário pode possuir apenas uma assinatura cadastrada, e cada assinatura pertence a um único usuário. Essa restrição é garantida pelo atributo id_usuario, definido como chave estrangeira e como único na tabela assinatura.

Relacionamento entre Artista e Álbum (1:N)

O relacionamento entre as tabelas artista e album é do tipo um para muitos (1:N). Um artista pode lançar diversos álbuns ao longo da carreira, enquanto cada álbum pertence a apenas um artista. Esse relacionamento é implementado pela chave estrangeira id_artista presente na tabela album.

Relacionamento entre Álbum e Música (1:N)

A relação entre album e musica também é do tipo um para muitos (1:N). Cada álbum pode conter várias músicas, mas cada música está associada a um único álbum. Esse vínculo é realizado por meio da chave estrangeira id_album na tabela musica.

Relacionamento entre Gênero e Música (1:N)

O relacionamento entre genero e musica é do tipo um para muitos (1:N). Um mesmo gênero musical pode classificar diversas músicas, enquanto cada música pertence a apenas um gênero cadastrado na plataforma. Esse relacionamento é implementado pela chave estrangeira id_genero presente na tabela musica.

Relacionamento entre Usuário e Playlist (1:N)

Entre as tabelas usuario e playlist existe um relacionamento um para muitos (1:N). Um usuário pode criar várias playlists, mas cada playlist pertence a apenas um usuário. A associação é realizada através da chave estrangeira id_usuario armazenada na tabela playlist.

Relacionamento entre Playlist e Música (N:N)

O relacionamento entre playlist e musica é do tipo muitos para muitos (N:N). Uma playlist pode conter diversas músicas e uma mesma música pode estar presente em várias playlists. Para implementar esse relacionamento foi criada a tabela associativa playlist_musica, composta pelas chaves id_playlist e id_musica, além do atributo data_adicao, que registra quando a música foi adicionada à playlist.

Relacionamento entre Usuário, Música e Histórico de Reprodução

A tabela historico_reproducao é responsável por registrar as reproduções realizadas na plataforma. Ela estabelece relacionamentos com as tabelas usuario e musica, permitindo armazenar qual usuário reproduziu determinada música e em qual momento a reprodução ocorreu. Dessa forma, um usuário pode possuir diversos registros de reprodução e uma música pode ser reproduzida inúmeras vezes por diferentes usuários.

Em conjunto, esses relacionamentos possibilitam representar de forma consistente as operações da plataforma SoundWave, garantindo integridade referencial e permitindo a realização de consultas analíticas e operacionais sobre usuários, artistas, músicas, playlists e reproduções.

DECISÕES TÉCNICAS

Durante o desenvolvimento do banco de dados da plataforma, foram adotadas diversas decisões técnicas com o objetivo de garantir a integridade, a consistência e a eficiência no armazenamento das informações. A modelagem foi baseada em boas práticas de banco de dados relacionais, utilizando chaves primárias e estrangeiras, restrições de integridade e tipos de dados adequados para cada necessidade do sistema.

Chaves Primárias

Todas as tabelas do banco possuem uma chave primária definida por identificadores numéricos inteiros com a propriedade `AUTO_INCREMENT`. Essa abordagem foi escolhida por garantir unicidade para cada registro, além de simplificar os relacionamentos entre as tabelas e melhorar o desempenho das operações de consulta e junção. Exemplos dessa estratégia podem ser observados nos campos `id_usuario`, `id_artista`, `id_album`, `id_musica` e demais identificadores do banco.

Chaves Estrangeiras

As chaves estrangeiras foram utilizadas para implementar os relacionamentos entre as entidades e garantir a integridade referencial dos dados. Dessa forma, não é possível cadastrar registros que façam referência a informações inexistentes. Por exemplo, uma música só pode ser cadastrada se estiver vinculada a um álbum existente, e uma playlist só pode ser criada para um usuário previamente cadastrado.

Tipos de Dados

Os tipos de dados foram escolhidos de acordo com a natureza das informações armazenadas. Campos numéricos de identificação utilizam o tipo `INT`, enquanto informações textuais, como nomes de usuários, artistas e títulos de músicas, utilizam `VARCHAR` com tamanhos adequados. Datas são armazenadas utilizando os tipos `DATE` e `DATETIME`, permitindo registrar tanto datas simples quanto informações completas de data e hora. Já valores monetários, como o preço da assinatura, utilizam o tipo `DECIMAL (8,2)`, garantindo precisão no armazenamento de valores financeiros.

Utilização de Campos NOT NULL

Os atributos considerados essenciais para o funcionamento do sistema foram definidos como NOT NULL, impedindo o armazenamento de registros incompletos. Informações como nome do usuário, e-mail, título da música, duração e tipo de plano são obrigatórias, pois são fundamentais para a identificação e utilização dos dados dentro da plataforma.

Por outro lado, alguns atributos foram mantidos como opcionais e podem receber valores nulos. É o caso de informações como pais_origem na tabela artista e data_lancamento na tabela album, que podem não estar disponíveis no momento do cadastro.

Restrições UNIQUE

Foram definidas restrições UNIQUE em campos que não podem possuir valores duplicados. O campo email da tabela usuario foi definido como único para evitar que dois usuários sejam cadastrados com o mesmo endereço eletrônico. Da mesma forma, o campo nome_artista da tabela artista também possui essa restrição, impedindo registros duplicados de artistas. Além disso, o atributo id_usuario da tabela assinatura foi definido como único para garantir o relacionamento um para um entre usuário e assinatura.

Valores Padrão (DEFAULT)

A utilização de valores DEFAULT foi adotada para automatizar o preenchimento de determinados atributos e reduzir erros de inserção de dados. A data de cadastro de usuários e artistas, bem como a data de criação das playlists, recebe automaticamente a data atual do sistema. O campo ativo da tabela usuario assume o valor verdadeiro por padrão, enquanto o status da assinatura é inicialmente definido como “ATIVA”. O histórico de reprodução e o registro de adição de músicas em playlists também recebem automaticamente a data e hora atuais por meio do comando CURRENT_TIMESTAMP.

Restrições CHECK

Foram implementadas restrições CHECK para validar regras de negócio diretamente no banco de dados. Na tabela musica, a restrição garante que a duração da música seja maior que zero, evitando registros inválidos. Na tabela assinatura, a restrição assegura que o valor mensal da assinatura seja igual ou superior a zero, impedindo o cadastro de valores negativos. Essas validações aumentam a confiabilidade dos dados armazenados.

Chave Primária Composta

Na tabela associativa playlist_musica foi utilizada uma chave primária composta pelos campos id_playlist e id_musica. Essa decisão impede que a mesma música seja cadastrada mais de uma vez na mesma playlist, além de representar corretamente o relacionamento muitos para muitos existente entre playlists e músicas.

ESTRATÉGIA DE POPULAÇÃO

Após a criação da estrutura do banco de dados, foi realizada a etapa de população das tabelas com dados fictícios, porém baseados em situações realistas encontradas em plataformas de streaming musical. O objetivo dessa estratégia foi possibilitar a execução e validação das consultas SQL desenvolvidas, além de representar diferentes cenários de utilização do sistema.

Foram cadastrados usuários com diferentes datas de registro e situações de atividade, permitindo simular uma base de clientes diversificada. Entre os registros existem usuários ativos e inativos, bem como usuários com e sem assinaturas cadastradas. Essa variedade foi importante para testar consultas relacionadas ao controle de assinaturas e à identificação de usuários sem plano ativo na plataforma.

A tabela de assinaturas foi preenchida com diferentes tipos de plano, como Básico, Premium e Família, além de registros com status distintos, incluindo assinaturas ativas e canceladas. Dessa forma, tornou-se possível representar diferentes perfis de utilização do serviço e realizar consultas relacionadas ao gerenciamento financeiro e ao acompanhamento dos assinantes.

Para compor o catálogo musical, foram cadastrados quinze artistas nacionais e internacionais de diferentes estilos musicais. Cada artista possui um álbum associado, e cada álbum contém várias músicas vinculadas a diferentes gêneros musicais, como Pop, Rock, Sertanejo, R&B, Indie Rock, Funk e Alternativo. Essa diversidade de informações permite realizar análises sobre preferências musicais, popularidade de gêneros e desempenho de artistas.

As playlists também foram populadas com cenários variados. Algumas playlists possuem grande quantidade de músicas, enquanto outras contêm poucas faixas. Além disso, uma playlist foi mantida propositalmente sem músicas associadas, possibilitando testar consultas que utilizam LEFT JOIN para identificar registros sem relacionamento correspondente.

Na tabela associativa `playlist_musica`, foram registradas músicas pertencentes a diferentes playlists, simulando o comportamento real dos usuários ao organizar

conteúdos musicais. Como uma mesma música pode estar presente em diversas playlists, os dados inseridos permitiram validar o relacionamento muitos para muitos existentes entre essas entidades.

O histórico de reproduções recebeu uma atenção especial por ser a principal fonte de dados para as consultas analíticas do projeto. Foram registrados diversos eventos de reprodução distribuídos entre diferentes usuários e músicas, em datas e horários distintos. Algumas músicas receberam várias reproduções, enquanto outras não foram reproduzidas nenhuma vez. Dessa forma, foi possível testar consultas relacionadas às músicas mais populares, músicas nunca reproduzidas, artistas mais reproduzidos e usuários mais ativos.

Além disso, a distribuição temporal dos registros de reprodução permitiu utilizar funções de janela, como LAG (), para analisar a sequência cronológica de utilização da plataforma pelos usuários. Também foi possível empregar funções de agregação e rankings para gerar informações estratégicas sobre o comportamento de consumo musical.

De modo geral, a população do banco foi planejada para contemplar diferentes situações de negócio, incluindo registros com e sem relacionamentos, usuários ativos e inativos, assinaturas em diferentes estados, músicas populares e não reproduzidas, além de playlists com diferentes quantidades de músicas. Essa abordagem garantiu uma base de testes mais completa e adequada para validar tanto as operações básicas quanto as consultas analíticas desenvolvidas para a plataforma SoundWave.

CONSULTAS DESENVOLVIDAS

Para atender aos objetivos do projeto, foram desenvolvidas treze consultas SQL utilizando recursos como junções entre tabelas, funções de agregação, filtros, subconsultas e funções de janela. Essas consultas permitem extrair informações relevantes sobre usuários, músicas, artistas, playlists e reproduções registradas na plataforma SoundWave.

Consulta 1: Músicas mais reproduzidas

Objetivo: identificar quais músicas possuem o maior número de reproduções na plataforma.

A consulta relaciona as tabelas musica e historico_reproducao, contabilizando a quantidade de reproduções de cada música por meio da função COUNT (). Os resultados são ordenados da música mais reproduzida para a menos reproduzida.

Consulta 2: Artistas com mais músicas cadastradas

Objetivo: verificar quais artistas possuem a maior quantidade de músicas cadastradas no catálogo.

A consulta relaciona as tabelas artista, album e musica, contabilizando o total de músicas pertencentes a cada artista.

Consulta 3: Quantidade de músicas por gênero musical

Objetivo: identificar quantas músicas existem em cada gênero musical cadastrado.

A consulta utiliza as tabelas genero e musica, permitindo inclusive a exibição de gêneros que não possuam músicas associadas.

Consulta 4: Playlists com maior quantidade de músicas

Objetivo: identificar quais playlists possuem mais músicas cadastradas.

A consulta utiliza as tabelas `playlist` e `playlist_musica`, contabilizando a quantidade de faixas presentes em cada `playlist`.

Consulta 5: Usuários sem assinatura cadastrada

Objetivo: localizar usuários que ainda não possuem uma assinatura vinculada à plataforma.

A consulta utiliza um `LEFT JOIN` entre as tabelas `usuario` e `assinatura`, retornando apenas os usuários sem registro de assinatura.

Consulta 6: Músicas nunca reproduzidas

Objetivo: identificar músicas que ainda não receberam nenhuma reprodução.

A consulta relaciona as tabelas `musica` e `historico_reproducao`, retornando apenas as músicas que não possuem registros no histórico de reprodução.

Consulta 7: Artistas com músicas reproduzidas

Objetivo: identificar quais artistas possuem músicas que já foram reproduzidas pelos usuários da plataforma.

A consulta percorre os relacionamentos entre `artista`, `álbum`, `música` e `histórico` de reprodução, exibindo apenas artistas que possuem reproduções registradas.

Consulta 8: Usuários mais ativos

Objetivo: verificar quais usuários realizaram a maior quantidade de reproduções.

A consulta relaciona as tabelas `usuario` e `historico_reproducao`, contabilizando o total de reproduções realizadas por cada usuário.

Consulta 9: Músicas com reproduções acima da média

Objetivo: identificar músicas cujo número de reproduções está acima da média geral da plataforma.

A consulta utiliza uma subconsulta para calcular a média de reproduções e aplica a cláusula HAVING para selecionar apenas as músicas que superam esse valor médio. Essa informação pode ser utilizada para destacar conteúdos com melhor desempenho.

Consulta 10: Usuários com playlists contendo músicas do gênero Pop

Objetivo: identificar usuários que possuem playlists contendo músicas classificadas no gênero Pop.

A consulta utiliza subconsultas e o operador IN para localizar usuários que criaram playlists com músicas desse gênero, permitindo análises relacionadas às preferências musicais dos usuários.

Consulta 11: Artistas presentes em playlists

Objetivo: identificar quais artistas possuem músicas adicionadas às playlists criadas pelos usuários.

A consulta relaciona as tabelas artista, album, musica, playlist_musica e playlist, exibindo os artistas que aparecem nas playlists cadastradas na plataforma.

Consulta 12: Ranking dos artistas mais reproduzidos

Objetivo: criar uma classificação dos artistas com base na quantidade total de reproduções de suas músicas.

A consulta emprega a função de janela RANK(), permitindo gerar um ranking ordenado dos artistas mais reproduzidos para os menos reproduzidos. Essa informação é útil para análises de popularidade e estratégias de recomendação.

Consulta 13: Evolução das reproduções por usuário

Objetivo: analisar o comportamento temporal dos usuários em relação às reproduções realizadas.

A consulta utiliza a função de janela LAG() para identificar a reprodução anterior de cada usuário, permitindo acompanhar a sequência cronológica de utilização da plataforma e identificar padrões de consumo musical.

As consultas desenvolvidas abrangem tanto necessidades operacionais quanto análises estratégicas, demonstrando a capacidade do banco de dados em transformar dados armazenados em informações úteis para a gestão da plataforma SoundWave. Foram utilizados conceitos de junções, agregações, subconsultas e funções analíticas avançadas, explorando diferentes recursos da linguagem SQL.

Consulta BÔNUS

A consulta relaciona as tabelas de usuários, reproduções, músicas, álbuns e artistas para identificar quem ouviu cada artista. Em seguida, utiliza COUNT () para contar as reproduções e RANK () para criar um ranking dos artistas mais ouvidos por cada usuário. Por fim, exibe apenas o artista que ficou em primeiro lugar no ranking de cada pessoa.

EXPLICAÇÃO DETALHADA DAS CONSULTAS MAIS COMPLEXAS

Consulta 9: Músicas com reproduções acima da média geral

O objetivo desta consulta é identificar quais músicas apresentam desempenho superior à média de reproduções da plataforma. Para isso, a consulta utiliza as tabelas `musica` e `historico_reproducao`, relacionadas pelo campo `id_musica`. Inicialmente, é realizado um `INNER JOIN` para associar cada música aos seus registros de reprodução. Em seguida, a função `COUNT ()` calcula quantas vezes cada música foi reproduzida. Os resultados são agrupados por música utilizando a cláusula `GROUP BY`.

Após calcular o total de reproduções de cada música, a consulta utiliza a cláusula `HAVING` para aplicar um filtro sobre os resultados agregados. Esse filtro compara a quantidade de reproduções de cada música com a média geral da plataforma.

A média é obtida por meio de uma subconsulta. Nessa etapa, uma consulta interna calcula a quantidade de reproduções de cada música individualmente utilizando `COUNT (*)` e `GROUP BY id_musica`. Posteriormente, uma consulta externa aplica a função `AVG ()` sobre esses valores para determinar a média geral de reproduções.

Por fim, somente as músicas cuja quantidade de reproduções é superior ao valor médio calculado são exibidas no resultado. Essa consulta é útil para identificar conteúdos com desempenho acima do padrão da plataforma, podendo apoiar sistemas de recomendação, campanhas promocionais e playlists de destaque.

Consulta 10: Usuários que criaram playlists com músicas do gênero Pop

Esta consulta tem como finalidade identificar os usuários que possuem playlists contendo músicas do gênero Pop. A consulta principal é executada sobre a tabela `usuario`, responsável por armazenar os dados dos usuários cadastrados.

O processo começa por uma subconsulta que analisa as tabelas `playlist`, `playlist_musica` e `musica`. Essas tabelas são relacionadas por meio de `INNER JOIN`, permitindo localizar todas as músicas presentes nas playlists criadas pelos usuários.

Em seguida, a consulta verifica quais dessas músicas pertencem ao gênero Pop. Para isso, utiliza uma segunda subconsulta que consulta a tabela `genero` e recupera o identificador correspondente ao gênero denominado “Pop”. Esse identificador é utilizado como critério de filtragem na consulta principal.

Após identificar as playlists que possuem músicas desse gênero, a subconsulta retorna os identificadores dos usuários responsáveis por essas playlists. O operador `IN` compara esses identificadores com os registros existentes na tabela `usuario`, retornando apenas os usuários que atendem à condição estabelecida.

Essa consulta possui importância estratégica porque permite identificar preferências musicais dos usuários e pode servir de base para mecanismos de personalização e recomendação de conteúdo.

Consulta 12: Ranking dos artistas mais reproduzidos

O objetivo desta consulta é construir um ranking dos artistas mais populares da plataforma com base na quantidade de reproduções de suas músicas. Para alcançar esse resultado, a consulta percorre toda a cadeia de relacionamentos entre as tabelas `artista`, `album`, `musica` e `historico_reproducao`.

Inicialmente, os relacionamentos são realizados por meio de `INNER JOIN`, permitindo associar cada reprodução registrada a uma música, seu álbum e, consequentemente, ao artista responsável. Depois disso, a função `COUNT()` contabiliza o total de reproduções acumuladas por cada artista.

A cláusula `GROUP BY` agrupa os registros pelo identificador e nome do artista, garantindo que o cálculo seja realizado individualmente para cada um deles. Em seguida, é aplicada a função de janela `RANK() OVER()`, que gera automaticamente uma classificação ordenada de acordo com a quantidade de reproduções obtidas.

A ordenação dentro da função é realizada em ordem decrescente, fazendo com que os artistas com mais reproduções ocupem as primeiras posições do ranking. Caso dois artistas apresentem a mesma quantidade de reproduções, ambos recebem a mesma colocação, característica própria da função RANK().

Os resultados produzidos por essa consulta permitem identificar os artistas de maior destaque na plataforma, fornecendo informações relevantes para decisões relacionadas a recomendações, campanhas de divulgação e análises de popularidade do catálogo musical.

Consulta 13: Evolução das reproduções por usuário

Esta consulta foi desenvolvida para analisar o comportamento dos usuários ao longo do tempo, permitindo visualizar a sequência das reproduções realizadas na plataforma. Para isso, são utilizadas as tabelas `usuario`, `historico_reproducao` e `musica`, relacionadas através de INNER JOIN. Dessa forma, cada registro exibe o usuário responsável pela reprodução, a música reproduzida e a data e horário da execução.

O principal diferencial desta consulta é a utilização da função de janela LAG(). Essa função permite acessar informações de uma linha anterior sem a necessidade de subconsultas ou operações adicionais de junção. Neste caso, a função retorna a data e hora da reprodução imediatamente anterior realizada pelo mesmo usuário.

Para garantir que a comparação ocorra apenas entre reproduções do mesmo usuário, é utilizada a cláusula PARTITION BY `u.id_usuario`, que separa os registros em grupos independentes. Em seguida, a cláusula ORDER BY `h.data_hora` organiza as reproduções em ordem cronológica dentro de cada grupo.

Quando a função LAG() é executada, ela procura o registro anterior dentro da mesma partição e retorna sua data e horário. Para a primeira reprodução de cada usuário não existe registro anterior, portanto o resultado retornado é nulo.

A análise produzida por essa consulta possibilita acompanhar a evolução do uso da plataforma, identificar padrões de navegação, frequência de utilização e

comportamento temporal dos usuários. Esse tipo de informação é especialmente útil para análises avançadas de engajamento e retenção de usuários.

Consulta BÔNUS

O objetivo desta consulta é identificar qual artista foi o mais ouvido por cada usuário da plataforma. Para isso, é necessário percorrer o caminho existente entre os usuários e os artistas por meio dos registros de reprodução armazenados no banco de dados. As tabelas utilizadas são `usuario`, `historico_reproducao`, `musica`, `album` e `artista`.

Inicialmente, a consulta realiza uma sequência de `INNER JOIN` para relacionar cada reprodução registrada ao respectivo artista. A tabela `historico_reproducao` informa qual usuário ouviu determinada música. Em seguida, a tabela `musica` permite identificar a música reproduzida, a tabela `album` indica o álbum ao qual essa música pertence e, por fim, a tabela `artista` identifica o responsável pela obra musical. Dessa forma, cada reprodução passa a estar associada simultaneamente a um usuário e a um artista.

Após estabelecer os relacionamentos, a função `COUNT (*)` é utilizada para calcular quantas vezes cada usuário ouviu músicas de cada artista. Para que essa contagem seja feita corretamente, a consulta utiliza a cláusula `GROUP BY`, agrupando os registros por usuário e por artista. Como resultado dessa etapa, obtém-se uma tabela contendo cada combinação usuário-artista acompanhada do total de reproduções correspondentes.

Na etapa seguinte, é aplicada a função de janela `RANK () OVER ()`. Essa função cria uma classificação dos artistas para cada usuário com base na quantidade de reproduções. A cláusula `PARTITION BY u.id_usuario` faz com que cada usuário possua seu próprio ranking individual, enquanto `ORDER BY COUNT (*) DESC` organiza os artistas do mais ouvido para o menos ouvido.

O resultado dessa classificação é armazenado em uma subconsulta. Nela, cada artista recebe uma posição dentro do ranking de cada usuário. O artista mais ouvido recebe a posição 1, o segundo mais ouvido recebe a posição 2 e assim

sucessivamente. Caso dois artistas possuam exatamente a mesma quantidade de reproduções para um determinado usuário, ambos receberão a mesma posição devido ao comportamento da função RANK ().

Por fim, a consulta externa utiliza a condição WHERE posicao = 1 para selecionar apenas os registros que ocupam a primeira posição do ranking. Dessa forma, o resultado exibe o nome do usuário, o artista mais escutado por ele e o total de reproduções desse artista. Em situações de empate, todos os artistas empatados na primeira colocação serão apresentados.

CONCLUSÃO

O desenvolvimento do banco de dados da plataforma SoundWave permitiu aplicar, na prática, os principais conceitos estudados em bancos de dados relacionais, desde a etapa de modelagem até a implementação das consultas SQL. Ao longo do projeto, foi possível compreender a importância de uma estrutura bem planejada para garantir a organização, a integridade e a consistência das informações armazenadas.

Uma das principais dificuldades encontradas durante o desenvolvimento foi a definição adequada dos relacionamentos entre as entidades do sistema. Foi necessário analisar cuidadosamente as regras de negócio para identificar quando utilizar relacionamentos do tipo 1:1, 1:N e N:N, bem como a necessidade de criar uma tabela associativa para representar corretamente a relação entre playlists e músicas. Além disso, a construção das consultas mais avançadas exigiu atenção especial na utilização de subconsultas, funções de agregação e funções de janela.

Durante a implementação, algumas decisões foram revistas para melhorar a qualidade da modelagem. Optou-se pela utilização de identificadores numéricos como chaves primárias em todas as tabelas, mantendo atributos como e-mail e nome do artista apenas como campos com restrição de unicidade. Também foram incorporadas validações por meio de restrições CHECK, valores padrão com DEFAULT e regras de integridade referencial com chaves estrangeiras, tornando a estrutura mais robusta e segura.

Outro aprendizado importante foi a compreensão da importância da etapa de população do banco de dados. A criação de dados diversificados, incluindo usuários com e sem assinatura, playlists vazias, músicas reproduzidas e músicas sem reproduções, possibilitou testar diferentes cenários e validar corretamente as consultas desenvolvidas. Essa estratégia permitiu simular situações próximas às encontradas em ambientes reais de streaming musical.

A elaboração das consultas também contribuiu significativamente para o aprendizado do grupo. Foram utilizados recursos fundamentais da linguagem SQL, como JOIN, GROUP BY, HAVING, subconsultas e funções de agregação, além de

recursos mais avançados como as funções de janela RANK() e LAG(). A aplicação desses elementos permitiu transformar dados armazenados em informações úteis para análise de comportamento dos usuários, popularidade de artistas e desempenho das músicas da plataforma.

Por fim, o projeto atingiu os objetivos propostos ao desenvolver um banco de dados funcional para uma plataforma de streaming musical, contemplando modelagem, implementação, população e consulta dos dados. Além do aprendizado técnico sobre bancos de dados relacionais, a atividade proporcionou uma visão mais ampla sobre a importância dos dados na geração de informações estratégicas para apoio à tomada de decisões em sistemas reais.